CALCULADORA JUNIT – DAVID ANDREICA

Ejercicio 1

Crea una clase de pruebas llamada ***Calculadora + TusIniciales + Test***. En esta clase deberás replicar el método de prueba realizado anteriormente para la operación de suma (testSuma()) en la clase **CalculadoraTest**.

A partir de este momento, todos los ejercicios se realizarán sobre tu propia clase de pruebas.

Después debes crear los tests de los otros métodos de la clase Calculadora:

- testResta()
- testMultiplica()
- testDivide()

Ten en cuenta que los métodos resta() y divide() contienen errores introducidos de forma intencionada, por lo que algunas de las pruebas deberán fallar inicialmente.

Estos son los test iniciales que usaré para los métodos, por ahora, como menciona el enunciado, el método de resta y de divide tienen errores, así que voy a enseñar los nuevos métodos corregidos.

```java
@org.junit.jupiter.api.Test
void suma() {
    Calculadora calculadora = new Calculadora( primerNumero: 3, segundoNumero: 3);
    int resultado = calculadora.suma();
    assertEquals( expected: 6, resultado);
}

@org.junit.jupiter.api.Test
void resta() {
    Calculadora calculadora = new Calculadora( primerNumero: 5, segundoNumero: 3);
    int resultado = calculadora.resta();
    assertEquals( expected: 2, resultado);
}

@org.junit.jupiter.api.Test
void multiplica() {
    Calculadora calculadora = new Calculadora( primerNumero: 5, segundoNumero: 3);
    int resultado = calculadora.multiplica();
    assertEquals( expected: 15, resultado);
}

@org.junit.jupiter.api.Test
void divide() {
    Calculadora calculadora = new Calculadora( primerNumero: 10, segundoNumero: 5);
    int resultado = calculadora.resta();
    assertEquals( expected: 2, resultado);
}
```

```
CalculadoraDARTest (org.ex 23 ms
    divide()  17 ms
    suma()  5 ms
    resta()  1 ms
    multiplica()

2 tests failed, 2 passed  4 tests total, 23 ms
/Users/david/Library/Java/JavaVirtualMachines/openjdk-25.0.1/Contents/Home/bin/java ...

org.opentest4j.AssertionFailedError:
Expected :2
Actual   :15
<Click to see difference>

<6 internal lines>
    at org.example.CalculadoraDARTest.divide(CalculadoraDARTest.java:32)


org.opentest4j.AssertionFailedError:
Expected :2
Actual   :8
<Click to see difference>

<6 internal lines>
    at org.example.CalculadoraDARTest.resta(CalculadoraDARTest.java:18)


Process finished with exit code 255
```

Método resta corregido

```
public int resta() {  2 usages
    // Error introducido de forma intencionada para detectarlo con JUnit
    int resultado = primerNumero - segundoNumero;
    return resultado;
}
```

Método divide corregido

```
public int divide() {  no usages
    // Error introducido de forma intencionada para detectarlo con JUnit
    int resultado = primerNumero / segundoNumero;
    return resultado;
}
```

Ahora, todos los test son correctos

```
CalculadoraDARTest (org.ex 28 ms
    divide()  17 ms
    suma()  9 ms
    resta()  1 ms
    multiplica()  1 ms
```

Ejercicio 2

Hacer las modificaciones oportunas en el código para que salte un fallo en testDivideExcepcion()

Añadimos una excepción al método divide para cuando el segundo número sea cero.

```
public int divide() {  1 usage
    // Error introducido de forma intencionada para detectarlo con JUnit
    if (segundoNumero == 0) {
        throw new ArithmeticException("Error. No se puede dividir entre 0");
    } else {
        int resultado = primerNumero / segundoNumero;
        return resultado;
    }
}
```

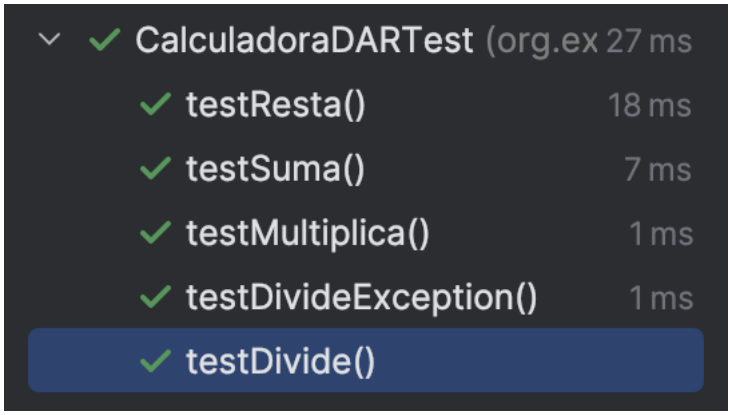
Test Junit para la división entre 0, comprobando que salta correctamente.

```
@Test
void testDivideException() {
    Calculadora calculadora = new Calculadora( primerNumero: 10, segundoNumero: 0);
    String valorEsperado = "Error. No se puede dividir entre 0";
    String valorDevuelto = "";

    try {
        calculadora.divide();
    } catch (ArithmeticException e) {
        valorDevuelto = e.getMessage();
    }

    assertEquals(valorEsperado, valorDevuelto);
}
```

Comprobamos que todas las pruebas sean correctas:



Ejercicio 3

Modifica y ejecuta el resto de métodos de la clase de prueba comprobando que siguen funcionando correctamente.

Test usando beforeEach:

```
@BeforeEach
public void inicioPruebas() {
    calc = new Calculadora( primerNumero: 30, segundoNumero: 10);
}

@AfterEach
public void finPruebas() {
    calc = null;
}

@Test
void testSumaEach() {
    int resultado = calc.suma();
    assertEquals( expected: 40, resultado);
}
```

Test completados:

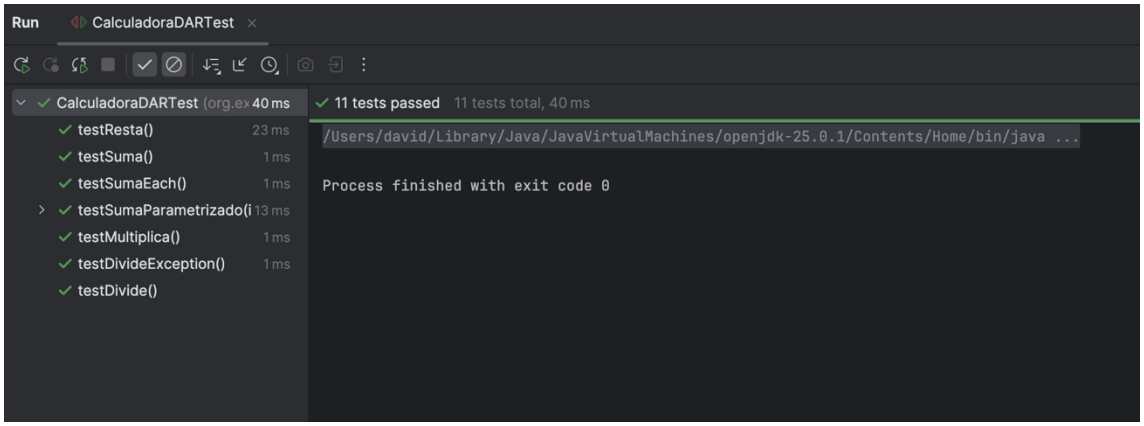
```
✓ CalculadoraDARTest (org.ex 22 ms
    ✓ testResta()            20 ms
    ✓ testSuma()
    ✓ testSumaEach()
    ✓ testMultiplica()
    ✓ testDivideException()   1 ms
    ✓ testDivide()            1 ms
```

Ejercicio 4

Crea y ejecuta pruebas en tu clase de pruebas para todos los métodos de la clase calculadora utilizando **@ValueSource** y **@CsvSource**

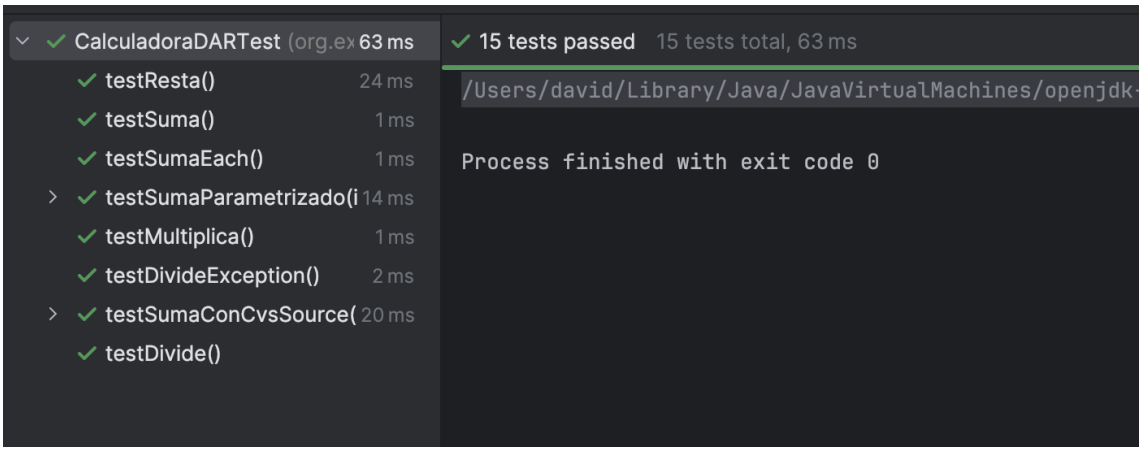
Usando ValueSource

```
@ParameterizedTest
@ValueSource(ints = {1,2,3,4,5})
void testSumaParametrizado(int num) {
    Calculadora calculadora = new Calculadora(num, segundoNumero: 3);
    int valorEsperado = num + calculadora.getSegundoNumero();
    int resultado = calculadora.suma();
    assertEquals(valorEsperado, resultado);
}
```



Usando CSVSource

```
@ParameterizedTest
@CsvSource({
        "1, 2, 3",
        "4, 5, 9",
        "-1, -2, -3",
        "0, 0, 0"
})
void testSumaConCvsSource(int a, int b, int valorEsperado) {
    Calculadora calculadora = new Calculadora(a, b);
    int resultado = calculadora.suma();
    assertEquals(valorEsperado, resultado);
}
```



Ejercicio 5

Muestra pantallazo de tu Código en donde se vea tu Suite de pruebas. La clase SuitePruebasCalc se debe llamar “SuitePruebas+TusIniciales” y debe contener tu clase de pruebas y la otra clase generada anteriormente.

No pude realizar el quinto punto debido a que al intentar usar las dependencias necesarias, me salían errores continuamente, como que me dejaba de detectar correctamente la versión de JUNIT, o hasta la de JAVA.

Ejercicio 6

Escribe una breve reflexión personal sobre qué te parece el concepto de pruebas unitarias.

En tu reflexión puedes comentar, por ejemplo:

- Qué utilidad crees que tienen en el desarrollo de software.
- Si consideras que ayudan a mejorar la calidad de los programas.
- Qué ventajas pueden aportar al trabajo de un desarrollador o a un equipo de desarrollo.

Investigación breve

Busca información sobre la pirámide de testing (Testing Pyramid) y explica brevemente:

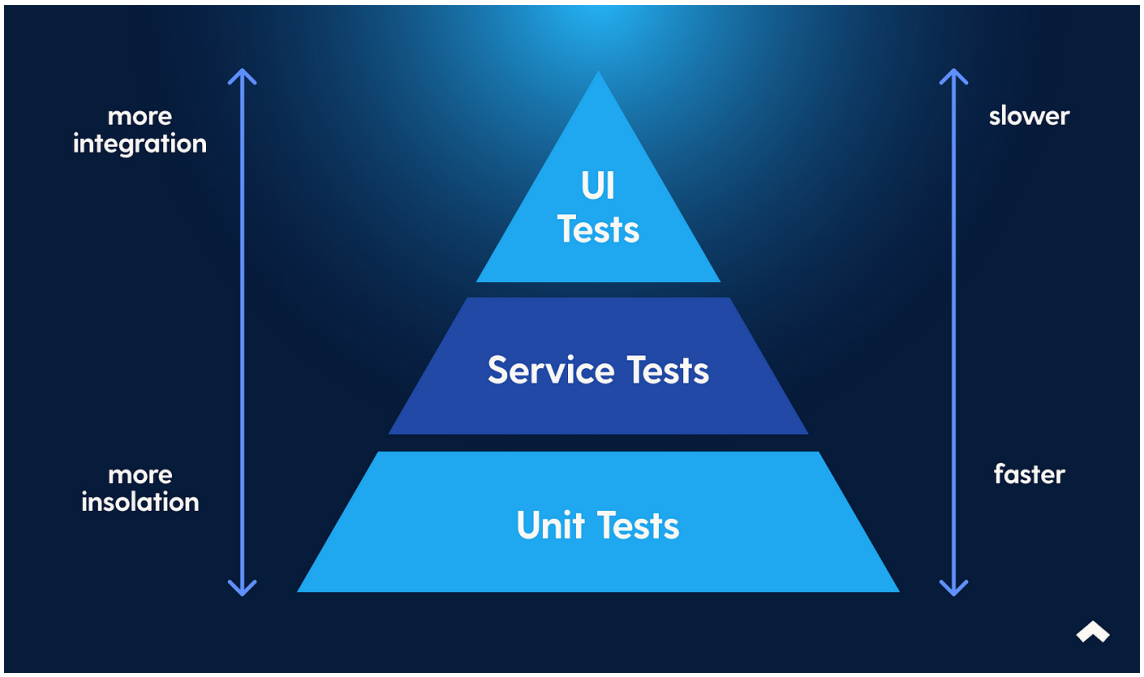
- Qué representa la pirámide.
- Qué tipos de pruebas aparecen en ella.
- Por qué las pruebas unitarias suelen situarse en la base.

La reflexión debe realizarse con tus propias palabras, independientemente del lenguaje de programación o del framework utilizado.

Para mí, las pruebas unitarias son muy útiles, sobre todo porque sirven para detectar de forma sencilla si tus métodos funcionan como deberían por separado. Gracias a los test unitarios puedes identificar si un test en concreto presenta errores, y corregirlo fácilmente.

Además, considero que el hacer métodos pensando en que después se harán test sobre estos, hace que hagamos un código más limpio y mejor pensado para que pase las pruebas unitarias. Al final, si no sabes ni cómo testear un método, significa que tan bien diseñado no está. Creo que ayuda a hacer mejor código.

Pirámide de testing:



¿Qué representa la pirámide?

La pirámide nos indica la cantidad de test que tenemos que hacer para cada tipo de prueba. Por ejemplo, para los test de UI, que en este caso diría que son los test de Sistema, se debe de probar que la aplicación completa funcione como se espera, lo que lo hace mucho más difícil de testear, ya que hace más complejo saber qué falla de todos los métodos que tiene la app.

Arriba del todo tenemos los test que menos debemos realizar, porque son los más complejos y lentos de realizar.

Después tenemos el punto medio, los test de servicios, que es en esencia, agrupar diversos métodos y comprobar que funcionen conjuntamente. No es tan complejo como los test del sistema, pero no es tan fácil detectar los problemas exactos como en los unitarios.

Abajo del todo tenemos el test que más debemos realizar: El unitario. Con estos test comprobamos método por método que funcione como se espera, permitiéndonos tener mucha más precisión para saber qué partes del sistema falla y cuales no. Estos test son los más rápidos de realizar.

¿Qué tipos de prueba aparecen en ella?

Test del sistema: Probar el sistema completo, con todos sus métodos

Test de integración: Comprueba que varios métodos juntos funcionan correctamente

Test unitarios: Comprueban que un método en concreto funciona como se espera